

Tài liệu Module SyntheticDataGenerator

Lê Hoàng An

May 31, 2025

1 Giới thiệu

Tài liệu này mô tả các luồng xử lý của class SyntheticDataGenerator, một lớp được thiết kế để tạo dữ liệu tổng hợp cho huấn luyện mô hình phân loại trạng thái khuôn mặt (Chú ý, Mất tập trung, Buồn ngủ). Dữ liệu được tạo ra với 6 đặc trưng hình học, mô phỏng các đặc trưng thực tế từ khuôn mặt. Class cung cấp các phương thức để tạo dữ liệu, trực quan hóa dữ liệu, và phân tích tương quan giữa các đặc trưng. Tài liệu bao gồm các luồng chính, vòng đời (lifecycle), luồng xác thực đầu vào, và các luồng chi tiết liên quan đến quá trình tạo và phân tích dữ liệu.

2 Flow Chính của Module

Module SyntheticDataGenerator hoạt động theo các bước chính sau:

1. **Khởi tạo:** Khởi tạo đối tượng với seed ngẫu nhiên và định nghĩa các phạm vi đặc trưng cho từng trạng thái (attention, distracted, sleepy).
2. **Tạo dữ liệu tổng hợp:** Tạo các mẫu dữ liệu với phân phối lớp được chỉ định, thêm nhiễu và correlation thực tế giữa các đặc trưng.
3. **Trực quan hóa dữ liệu:** Tạo các biểu đồ scatter để hiển thị phân phối của các cặp đặc trưng quan trọng.
4. **Phân tích tương quan:** Tạo ma trận tương quan tổng thể và theo từng lớp để phân tích mối quan hệ giữa các đặc trưng.

3 Lifecycle của Module

Vòng đời của một đối tượng SyntheticDataGenerator bao gồm các giai đoạn sau:

1. Khởi tạo (__init__):

- Nhận tham số seed để đảm bảo tính tái lập của dữ liệu ngẫu nhiên.
- Định nghĩa danh sách lớp(class_names): Chú ý, Mắt tập trung, Buồn ngủ.
- Định nghĩa phạm vi đặc trưng(feature_ranges) cho 6 đặc trưng: avg_ear, mar, head_pose_x, head_pose_y, face_ratio, eye_distance.

2. Tạo dữ liệu (generate_synthetic_data):

- Tạo số lượng mẫu theo phân phối lớp.
- Tạo vector đặc trưng cho từng mẫu dựa trên phạm vi đặc trưng và thêm correlation thực tế.
- Thêm nhiễu ngẫu nhiên và chuẩn hóa dữ liệu.
- Xáo trộn dữ liệu để đảm bảo tính ngẫu nhiên.

3. Trực quan hóa (visualize_data):

- Tạo các biểu đồ scatter cho các cặp đặc trưng quan trọng.
- Lưu biểu đồ nếu có đường dẫn save_path.

4. Phân tích tương quan (create_correlation_analysis):

- Tạo ma trận tương quan tổng thể và theo từng lớp.
- Hiển thị dưới dạng heatmap sử dụng seaborn.

5. Kết thúc: Không có phương thức hủy rõ ràng, đối tượng được giải phóng tự động bởi Python garbage collector khi không còn được tham chiếu.

4 Input Validation Flow

Luồng xác thực đầu vào được thực hiện như sau:

1. Khởi tạo (__init__):

- Kiểm tra seed: Đặt seed ngẫu nhiên bằng `np.random.seed` để đảm bảo tính tái lập.
- Không có kiểm tra rõ ràng cho tham số seed (ví dụ: kiểm tra kiểu dữ liệu).

2. Tạo dữ liệu tổng hợp (**generate_synthetic_data**):

- Kiểm tra `class_distribution`: Nếu tổng phân phối không bằng 100%, ném lỗi `ValueError`.
- Kiểm tra `n_samples`: Đảm bảo là số nguyên dương (mặc dù không có kiểm tra rõ ràng, lỗi runtime có thể xảy ra nếu âm).
- Kiểm tra `noise_level`: Không có kiểm tra rõ ràng, nhưng nhiều được áp dụng bằng phân phối chuẩn (`np.random.normal`).
- Đảm bảo giá trị đặc trưng hợp lý: Sử dụng `np.clip` để giới hạn giá trị đặc trưng trong khoảng `[0, 2]`.

3. Trực quan hóa dữ liệu (**visualize_data**):

- Kiểm tra `X` và `y`: Giả định `X` và `y` là các mảng NumPy hợp lệ, không có kiểm tra rõ ràng.
- Kiểm tra `save_path`: Nếu được cung cấp, lưu biểu đồ tại đường dẫn; không kiểm tra tính hợp lệ của đường dẫn (có thể gây lỗi nếu không hợp lệ).

4. Phân tích tương quan (**create_correlation_analysis**):

- Kiểm tra `X` và `y`: Giả định `X` và `y` là các mảng NumPy hợp lệ, không có kiểm tra rõ ràng.
- Giả định thư viện `pandas` và `seaborn` đã được cài đặt, không có xử lý lỗi cho trường hợp thiếu thư viện.

5 Các Flow Chi Tiết

Dưới đây là các luồng chi tiết liên quan đến các phương thức chính:

5.1 Tạo dữ liệu tổng hợp (**generate_synthetic_data**)

- **Đầu vào:** Số lượng mẫu (`n_samples`), phân phối lớp (`class_distribution`), cờ thêm nhiễu (`add_noise`), mức nhiễu (`noise_level`).

- **Xử lý:**

- Nếu `class_distribution` là `None`, sử dụng phân phối mặc định [40, 35, 25].
- Kiểm tra tổng phân phối bằng 100%, nếu không, ném `ValueError`.
- Tính số mẫu cho mỗi lớp, điều chỉnh lớp đầu tiên để đảm bảo tổng số mẫu bằng `n_samples`.
- Lặp qua từng lớp:
 - * Tạo mẫu với 6 đặc trưng bằng `np.random.uniform` trong phạm vi `feature_ranges`.
 - * Nếu trạng thái là `sleepy`, thêm `correlation`: Nếu `avg_ear` thấp, tăng `mar`.
 - * Nếu trạng thái là `attention`, làm mịn mẫu bằng cách kết hợp với giá trị trung bình của phạm vi.
 - * Nếu `add_noise` là `True`, thêm nhiễu Gaussian với `noise_level`.
 - * Giới hạn giá trị đặc trưng trong [0, 2] bằng `np.clip`.
- Xáo trộn dữ liệu bằng `np.random.shuffle`.

- **Đầu ra:** Mảng `X` (dữ liệu đặc trưng) và `y` (nhãn lớp).

5.2 Trực quan hóa dữ liệu (**visualize_data**)

- **Đầu vào:** Dữ liệu `X`, nhãn `y`, đường dẫn lưu tùy chọn `save_path`.

- **Xử lý:**

- Tạo 6 biểu đồ scatter cho các cặp đặc trưng quan trọng: `EAR vs MAR`, `Head Pose X vs Y`, `EAR vs Head Pose X`, `MAR vs Head Pose Y`, `Face Ratio vs Eye Distance`, `EAR vs Face Ratio`.
- Mỗi lớp được vẽ với màu khác nhau (Chú ý: xanh lá, Mắt tập trung: đỏ, Buồn ngủ: xanh dương).
- Thêm nhãn trục, tiêu đề, và lưới cho từng biểu đồ.
- Nếu `save_path` được cung cấp, lưu biểu đồ với độ phân giải 300 DPI.
- Hiển thị biểu đồ bằng `plt.show`.

- **Đầu ra:** Biểu đồ scatter hiển thị trên màn hình và/hoặc lưu tại `save_path`.

5.3 Phân tích tương quan (`create_correlation_analysis`)

- **Đầu vào:** Dữ liệu `X`, nhãn `y`.
- **Xử lý:**
 - Tạo DataFrame từ `X` với tên đặc trưng: `EAR`, `MAR`, `Head_Pose_X`, `Head_Pose_Y`, `Face_Ratio`, `Eye_Distance`.
 - Thêm cột `Class` dựa trên `y`.
 - Tạo heatmap cho ma trận tương quan tổng thể bằng `df[feature_names].corr`.
 - Tạo heatmap cho ma trận tương quan của từng lớp bằng cách lọc dữ liệu theo lớp.
 - Sử dụng `seaborn.heatmap` với màu `coolwarm` và định dạng số thập phân 2 chữ số.
 - Hiển thị các biểu đồ bằng `plt.show`.
- **Đầu ra:** Biểu đồ heatmap hiển thị ma trận tương quan tổng thể và theo từng lớp.

6 Kết luận

Class `SyntheticDataGenerator` cung cấp một giải pháp mạnh mẽ để tạo dữ liệu tổng hợp cho huấn luyện mô hình phân loại trạng thái khuôn mặt. Các luồng xử lý được thiết kế để tạo dữ liệu thực tế với correlation giữa các đặc trưng, hỗ trợ trực quan hóa và phân tích tương quan. Để sử dụng hiệu quả, cần đảm bảo các thư viện (`numpy`, `matplotlib`, `pandas`, `seaborn`) được cài đặt và các tham số đầu vào (như `class_distribution`, `n_samples`) được cấu hình hợp lý.